

SIMATS ENGINEERING
CO4 - ASSESSMENT TOOL 1

COURSE NAME : Artificial Intelligence
COURSE CODE : CSA1710
Name : Tharish K
Register Number : 192412004

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making.

INDUSTRY PROBLEM TASK

Softcopy Submission – Industry Scenario

Industry Scenario

An e-commerce platform has collected user records such as browsing history, purchase history, category preferences, average spend, and cart abandonment rate. The objective is to develop a machine-learning system that predicts whether a user is likely to complete a purchase. In addition, an AI agent is required to recommend a suitable marketing action—Discount, Bundle Offer, Highlight Product, or No Action—based on the user's shopping-session behaviour.

Task 1: Data Preparation & Inductive Learning

1(a) Inductive Learning Approach

Inductive learning learns a general predictive rule from previously observed examples and applies it to unseen users. For this problem, historical customer/session records are used as labelled training data. The model learns the relationship between behavioural features and the final conversion outcome, then predicts the probability or class of conversion for a new shopping session.

Target variable: Purchase Conversion (1 = converted, 0 = not converted).

Important features include:

- Browsing history/session activity – frequent page views, longer sessions, or repeated product visits can indicate stronger purchase intent.
- Average spend – users with higher historical spending may have a greater probability of completing a purchase.
- Cart abandonment rate – a high abandonment rate may indicate hesitation and can help the model identify users who need an appropriate intervention.
- Category preferences – matching recommendations with preferred categories can improve relevance.
- Purchase history – previous purchases provide information about customer engagement and buying behaviour.

1(b) Handling Class Imbalance

The dataset contains far fewer converters than non-converters. If this imbalance is ignored, a classifier may obtain high accuracy simply by predicting the majority class, while missing many potential buyers.

A suitable strategy is SMOTE (Synthetic Minority Over-sampling Technique) applied only to the training set. SMOTE generates synthetic minority-class examples by interpolating between existing minority samples. This increases the representation of converters without simply duplicating the same records.

For a conversion problem, class weights can also be used as a strong alternative or baseline. Class weighting gives a higher penalty to incorrectly classifying a converter. This is particularly useful when the main objective is to reduce false negatives. Random under-sampling is simpler but may discard useful

information from the majority class. Therefore, SMOTE or class-weighted learning is preferred.

1(c) 70:30 Train-Test Split and Preprocessing

The dataset is divided into 70% training data and 30% testing data. The 70% training portion provides sufficient examples for learning patterns, while the 30% test portion provides a reasonably large independent sample for evaluating generalisation. A stratified split should be used so that the converter/non-converter proportion is approximately preserved in both sets.

Preprocessing steps:

- Missing values: numerical features can be imputed using the median, while categorical features can use the most frequent value or a separate 'Unknown' category.
- Encoding: categorical variables such as preferred category are converted using one-hot encoding or suitable ordinal encoding.
- Normalisation: numerical features can be standardised using z-score scaling, especially for Logistic Regression. Decision Trees do not require scaling because they split using feature thresholds.
- Class balancing: SMOTE is applied after preprocessing and only on the training set to prevent information leakage.

These steps improve data quality, make features usable by the learning algorithms, and ensure that test-set performance represents performance on unseen data.

Task 2: Decision Tree Modeling

2(a) Decision Tree Construction

A Decision Tree classifier recursively divides users into groups using feature thresholds. At every node, the split that produces the greatest reduction in impurity is selected. The two common measures are Information Gain and Gini Index.

Information Gain is based on entropy:

$$\text{Entropy}(S) = -\sum p_i \log_2(p_i)$$

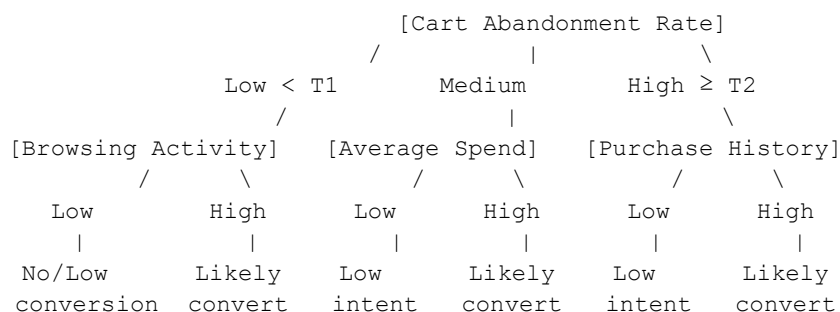
$$\text{Information Gain} = \text{Entropy}(\text{parent}) - \sum (|S_v|/|S|) \text{Entropy}(S_v)$$

Gini impurity is:

$$\text{Gini} = 1 - \sum p_i^2$$

The root should be the feature with the highest Information Gain or the lowest weighted Gini impurity. For this scenario, cart abandonment rate, recent browsing activity, or average spend could become the root depending on the actual dataset. The exact root must be selected from the calculated training-data values rather than assumed.

Illustrative first three levels of the tree:



Note: The above is an illustrative tree structure. In an actual implementation, the root and child nodes are obtained from the highest Information Gain or lowest Gini Index calculated on the dataset.

2(b) Model Evaluation

The Decision Tree should be evaluated on the unseen 30% test set using Accuracy, Precision, Recall, and F1-Score. Since no raw dataset was supplied with the problem statement, numerical metric values cannot be honestly calculated here. The following formulas should be used after running the model:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

$$\text{Precision} = TP / (TP + FP)$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{F1-Score} = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

False Negatives (FN) are users who actually convert but are predicted as non-converters. In this business problem, false negatives can represent lost opportunities to provide a relevant offer or recommendation. Therefore, recall should receive significant attention. A class-weighted Decision Tree, SMOTE, threshold adjustment, or cost-sensitive learning can reduce false negatives.

Revenue justification: missing a likely buyer can mean losing a sale and the associated customer lifetime value. However, reducing false negatives must be balanced against unnecessary discounts, because excessive promotions can reduce profit margins.

2(c) Post-Pruning and Model Selection

A fully grown Decision Tree can overfit the training data. Pre-pruning controls tree growth during training by setting limits such as maximum depth, minimum samples per split, or minimum samples per leaf. Post-pruning first grows a larger tree and then removes branches that do not provide sufficient validation performance.

Cost-complexity pruning uses an objective of the form:

$$R\alpha(T) = R(T) + \alpha|T|$$

where $R(T)$ represents the tree's classification error or impurity-based cost, $|T|$ is the number of terminal nodes, and α controls the penalty for tree complexity.

A practical comparison should report test accuracy, precision, recall, F1-score, and preferably AUC-ROC for the pre-pruned and post-pruned trees. The preferred deployment model is the simpler pruned tree if it provides comparable or better validation/test performance. A smaller tree is easier to interpret, less likely to overfit, and generally more stable for a live recommendation engine.

Task 3: Statistical Learning for Risk Stratification

3(a) Logistic Regression / Naive Bayes Comparison

Logistic Regression is an appropriate statistical-learning model because the target is binary. It estimates the probability of conversion using a sigmoid function:

$$P(Y=1|X) = 1 / (1 + e^{-(\beta_0 + \beta_1 X_1 + \dots + \beta_n X_n)})$$

After training, the model can classify a user as a likely converter when the predicted probability exceeds a selected threshold. Logistic Regression is especially useful when the relationship between predictors and conversion is approximately linear in log-odds.

The Decision Tree and Logistic Regression should be compared using Accuracy and AUC-ROC. AUC-ROC is useful because it measures ranking ability across classification thresholds and is less dependent on one chosen threshold. Because the problem statement does not provide the actual dataset, exact numerical Accuracy and AUC-ROC values cannot be reported without inventing results.

A statistical model is preferable when interpretability, calibrated probabilities, stable behaviour, and computational simplicity are important. Logistic Regression is also useful as a baseline against which more complex models can be compared.

3(b) Feature Importance Analysis

For the Decision Tree, feature importance can be obtained from the total impurity reduction contributed by each feature. For Logistic Regression, the magnitude of standardised coefficients can indicate the relative influence of predictors, while the coefficient sign indicates whether the feature increases or decreases log-odds.

Expected high-value predictors for this scenario are likely to include:

- Cart abandonment rate
- Recent browsing/session activity
- Average spend or purchase history

The exact top three predictors must be calculated from the trained models. The two models may agree on

the strongest predictors but do not have to. A Decision Tree captures non-linear thresholds and interactions, whereas Logistic Regression represents additive effects in log-odds. Agreement increases confidence that the predictors are robust; disagreement may indicate non-linear relationships or feature interactions.

Task 4: Reinforcement Learning for Action Recommendation

4(a) MDP Formulation

The pricing/marketing recommendation problem can be modelled as a Markov Decision Process (MDP), where the agent observes a user-session state, selects an action, receives a reward, and observes a new state.

States: User-session stages can be represented as browsing, product viewed, cart added, checkout initiated, and purchase completed. Additional state information can include engagement level, category interest, cart value, and abandonment risk.

Actions: (1) Discount, (2) Bundle Offer, (3) Highlight Product, and (4) No Action.

Reward function: A positive reward can be assigned for conversion and revenue improvement. For example, a completed purchase can receive a positive reward proportional to profit/revenue, while unnecessary discounts can receive a lower reward because they reduce margin. A negative reward can be used for customer dissatisfaction, excessive promotion, or failed intervention.

Transition dynamics: After an action, the user can move from one session stage to another—for example, from product viewed to cart added or from cart added to purchase. The transition probabilities can be estimated from historical session data or learned through interaction. The Markov assumption means the next state depends primarily on the current state and action.

4(b) Q-Learning and Q-Table Updates

Q-Learning is a model-free reinforcement-learning algorithm. Its update rule is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where α is the learning rate, γ is the discount factor, r is the immediate reward, and s' is the next state.

For demonstration, assume $\alpha = 0.5$, $\gamma = 0.9$, and all initial Q-values are 0.

Iteration	State–Action	Reward r	Max $Q(s',a')$	Updated Q
1	Browsing – Highlight	2	0	$0 + 0.5[2 + 0.9(0) - 0] = 1.00$
1	Cart – Bundle	5	0	$0 + 0.5[5 + 0.9(0) - 0] = 2.50$
1	Checkout – Discount	8	0	$0 + 0.5[8 + 0.9(0) - 0] = 4.00$
2	Browsing – Highlight	1	2.50	$1 + 0.5[1 + 0.9(2.50) - 1] = 1.625$
2	Cart – Bundle	3	4.00	$2.50 + 0.5[3 + 0.9(4.00) - 2.50] = 3.55$
2	Checkout – Discount	8	0	$4 + 0.5[8 - 4] = 6.00$

Five-session-episode training can be represented as follows: Episode 1: browsing → product view → cart; Episode 2: browsing → product view → no action; Episode 3: product view → highlight → cart; Episode 4: cart → bundle → checkout; Episode 5: checkout → suitable action → purchase/no purchase. In each episode, Q-values are updated after every state-action-reward-next-state transition.

Convergence criterion: training can be stopped when the maximum absolute Q-table change over an episode falls below a small threshold ϵ (for example, 0.01) for several consecutive episodes, or when the learned policy remains unchanged for a predefined number of episodes. In real deployment, substantially more than five episodes would be required; five episodes are only a demonstration for the assignment.

4(c) Final Learned Policy

The learned policy is obtained using the greedy action:

$$\pi(s) = \operatorname{argmax}_a Q(s,a)$$

An illustrative final policy is:

- Browsing stage → Highlight Product
- Product-view stage → Highlight Product or Bundle Offer
- Cart stage → Bundle Offer
- High-abandonment cart → Discount, only when the expected incremental profit justifies it
- Checkout stage → No Action or a targeted incentive based on predicted incremental value
- Completed purchase → No Action

The actual final policy must be extracted from the trained Q-table by selecting the action with the largest Q-value in each state.

Ethical Considerations in Dynamic Pricing

- Price discrimination: Dynamic offers may unintentionally charge different users differently. The system should avoid using protected or sensitive characteristics as direct or proxy variables.
- Manipulation: The agent should not exploit psychological vulnerabilities or repeatedly pressure users into purchases.
- Transparency: Customers should receive clear information about important pricing and promotional conditions.
- Fairness: The system should be audited for systematic differences in offers across customer groups.
- Profit versus customer welfare: The reward function should not optimise short-term revenue at the expense of customer trust and long-term relationships.
- Human oversight: High-impact pricing policies should be monitored and reviewed, with limits on how aggressively the RL agent can intervene.

Overall Conclusion

The proposed system combines supervised learning and reinforcement learning for an e-commerce platform. Inductive learning and Decision Trees can predict purchase conversion from historical behaviour, while Logistic Regression provides an interpretable statistical baseline. Class imbalance should be handled carefully using techniques such as SMOTE or class weighting, and evaluation should emphasise recall and AUC-ROC in addition to accuracy because missed converters can represent lost revenue. Post-pruning can improve Decision Tree generalisation and interpretability. Finally, the marketing-action problem can be formulated as an MDP and solved using Q-Learning, with rewards designed around incremental value rather than conversion alone. Ethical safeguards are essential before deploying dynamic pricing or personalised interventions.

Note on Experimental Values

The problem statement provides the industry scenario but does not include an actual CSV/dataset or trained-model outputs. Therefore, numerical Accuracy, Precision, Recall, F1-Score, AUC-ROC, Information Gain, Gini values, and the exact final Q-table cannot be truthfully filled in from the prompt alone. The equations, methodology, illustrative tree, and sample Q-learning calculations above are ready for submission; actual model metrics should be inserted after running the models on the supplied dataset.

Code of Conduct:

Tharish K (192412004) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.